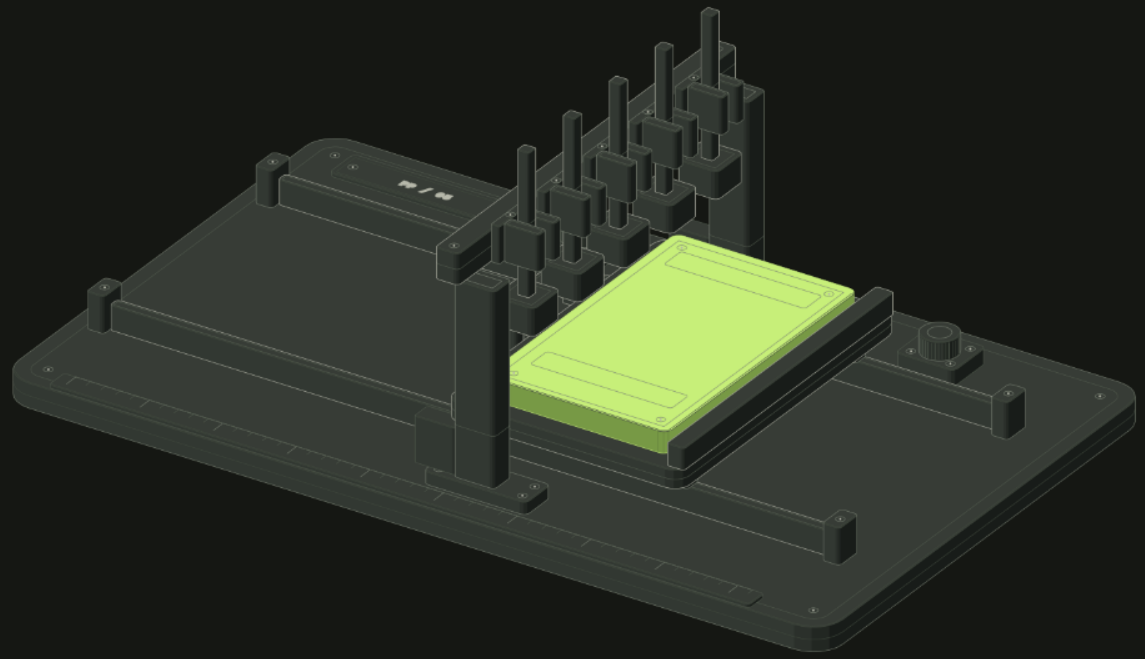
The fix.
And the proof.

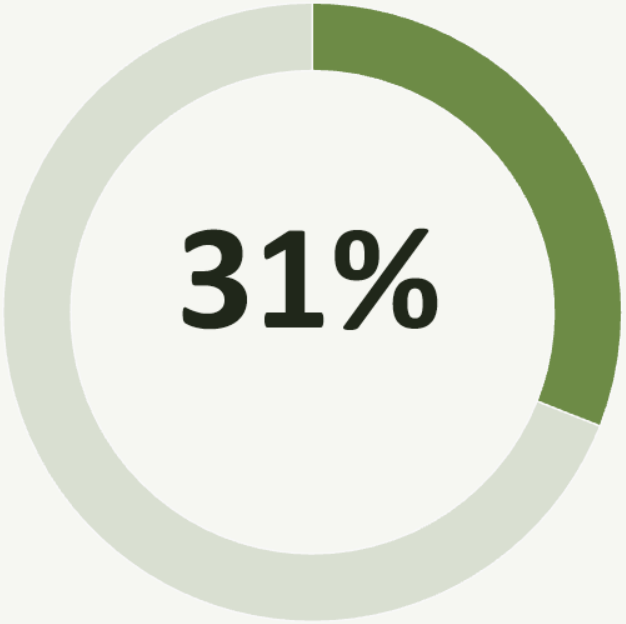


Security remediation that a reviewer can actually verify.

Scout → **Challenger** → **Fixer**

Hackathon / Checkpoint 1

An alert starts the work.



of breaches began with vulnerability exploitation

- **Triage** — Does this affect our app?
- **Reproduce** — Can we show the failure?
- **Repair** — Will it break normal behavior?
- **Review** — What evidence can we trust?

Verizon 2026 DBIR, using 2025 incident data. Workflow pain still needs customer validation.

A small team. A real review problem.

5–30

developers

TypeScript / Node SaaS
CTO buys. Developer uses.
No dedicated AppSec engineer.

```
GET /invoices/inv_1042
```

Signed in as Tenant B

200

Tenant A’s invoice is returned.

The reviewer needs to prove the boundary holds.

Synthetic example. Initial customer segment and pain are hypotheses, not interview findings.

Three roles. One evidence gate.

1 **Scout**
Find relevant risk

→

2 **Challenger**
Freeze the failing check

→

3 **Fixer**
Propose a scoped patch

- ✓ **Baseline fails**
- ✓ **Candidate passes**
- ✓ **Benign controls pass**
- ✓ **Test IDs match**
- ✓ **Commit IDs match**

A missing or failed condition blocks the draft. A human makes the merge decision.

Block the attack. Preserve the customer.

Security replay

`Tenant B → Tenant A invoice`

200 → **404**

Unauthorized request denied

Benign regression

`Invoice owner → own invoice`

200 → **200**

Legitimate access preserved

If either check fails, hold the patch.

Illustrative fixture results. This demonstrates a regression workflow, not comprehensive pentesting.

Focus on the last mile of security review.

Existing product	Documented capability
Dependabot	Dependency security-update PRs
GitHub Agentic Autofix	Code fixes, CodeQL rerun, draft PR
Snyk Agent Fix	Agentic remediation for code findings
Aikido Infinite	Continuous testing, AutoFix, retesting

Our proposed focus Portable tenant-boundary tests + customer-owned execution

Official vendor descriptions; full source links in notes. Product advantage remains unproven.

Price the outcome. Bound the compute.

₹4,999

per workspace / month

3 repositories
30 bounded validation runs

Monthly assumption	Base case	Cost stress
Run cost	₹25	₹100
Total variable cost	₹1,250	₹3,500
Contribution	75%	30%

30 runs + ₹500 infrastructure

Pricing hypothesis

Illustrative economics; excludes support, salaries, tax, payment fees and sales.

Earn three pilots before expanding.

10 → **5** → **3** → **2**

Interviews

Understand the last alert

Observations

Watch the real work

Scoped pilots

Use owned fixtures

Paid commitments

Test willingness to pay

Start with incubators and SaaS communities.

Proposed validation targets. No interviews, pilots, paid commitments or partnerships have been obtained.

Measure the work a developer gets back.

30%

less active remediation time

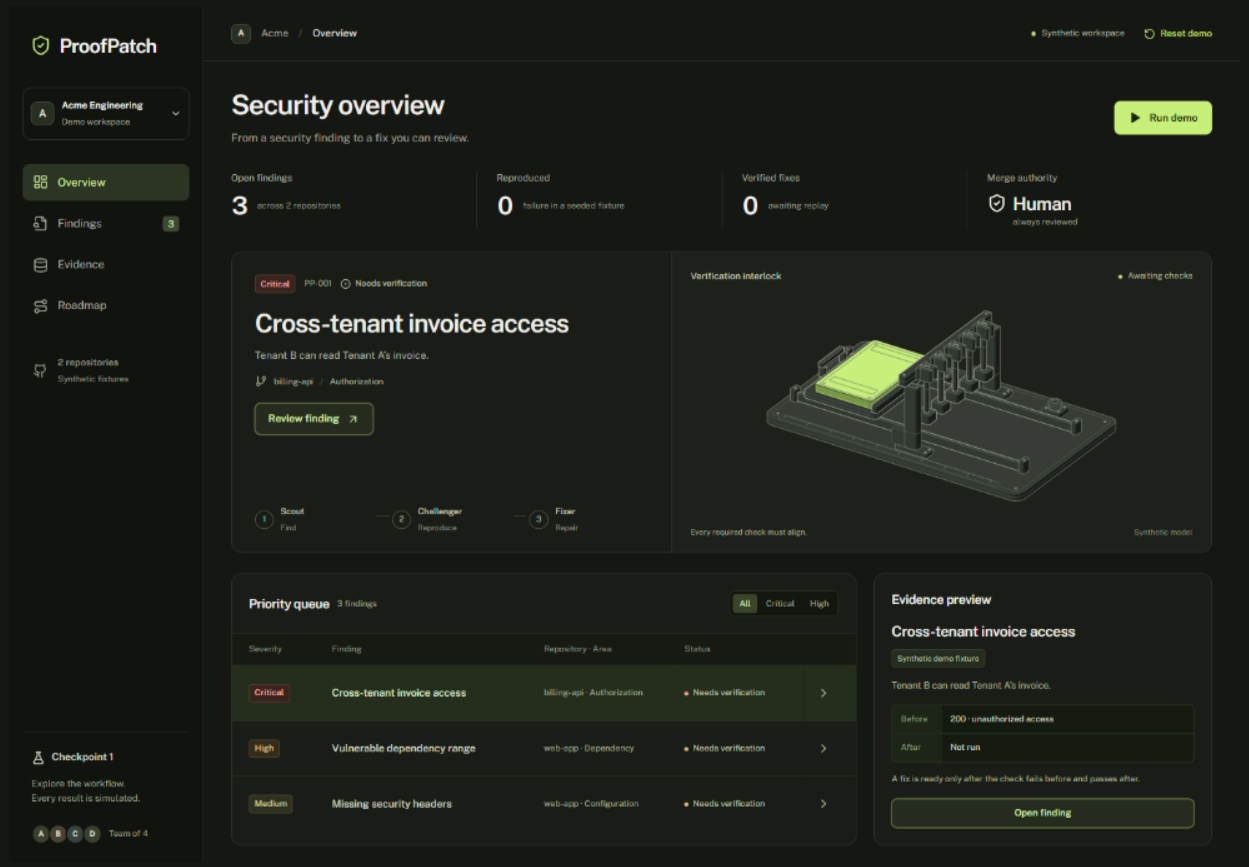
Target on matched pilot tasks.
Benchmark against the current workflow.

Trust before scale

- ✓ No bad-patch promotion on the controlled benchmark
- ✓ Evidence a reviewer can replay
- ✓ Costs measured with retries included

Targets, not results. Scenario: 100 workspaces × ₹4,999 × 12 = ₹59.988 lakh annual revenue; not market size.

The workflow is ready to demonstrate.



Built for Checkpoint 1

Interactive review
Blocked regression path
Local draft + evidence export

Next: one real loop

Scanner → isolated fixture
→ tested patch → draft PR

Four-member team. Current outcomes are synthetic. The 48-hour MVP window remains provisional.

Keep execution inside a bounded runner.

Reviewer
Next.js UI

→

Orchestrator
Hono + bounded jobs

→

Runner
Customer-owned fixture

- **Pinned commits**
- **Frozen test identity**
- **Scoped tokens**
- **Run budgets**
- **Human merge review**

Postgres records + scrubbed evidence artifacts

Planned architecture. Only the reviewer frontend is implemented today.

Evidence behind the idea.

Verizon 2026 DBIR	Breach context, based on 2025 data
GitHub Dependabot + Agentic Autofix	Security updates, code fixes and verification
Snyk Agent Fix	Agentic remediation documentation
Aikido Infinite	Continuous testing, fixes and retesting
OSV-Scanner + ZAP baseline	Tool options and passive-scan limits

Synthetic prototype. Real customer validation comes next.

Full URLs in speaker notes and PRD. Prices, savings and adoption targets are hypotheses.